

Devnet Status & Proof

What RoundFi actually ran on-chain — exercised end-to-end with real USDC

RoundFi

2026-06-23

CONTENTS

1. The claim, stated plainly
2. The deployed programs
3. The validated capabilities
4. Why "seven of eight" is the honest count
5. How to verify this yourself

Derived document. This is a derivation of [docs/spec/MASTER-SPEC.md](#) (§11 Deployed state & validated capabilities, with supporting context from §4 Protocol). The Master Spec is the single source of truth; if a number, address, or pool id here disagrees with it, the Master Spec wins. Every program ID and capability is pinned to the deployed state **as of 2026-06-12**. Transaction signatures live in the runbook ([docs/operations/v52-devnet-runbook.md](#)), not here.

1. The claim, stated plainly

Everything below was exercised **end-to-end on devnet on 2026-06-12 with real USDC**. Not a mock, not a unit test, not a slide: live programs, real token transfers, real on-chain attestations, across three real pools. Seven of the eight protocol capability areas were run to completion; the eighth — default settlement — is **armed and time-gated**, waiting only on a grace window to elapse before it can be replayed.

This document exists for one reader: a juror or partner who wants **proof the protocol is real and live**, not a deck describing one that might be. The way to read it is to take any row in §3, find the pool it was exercised on, and pull the matching transaction signature from the runbook. Every claim is reproducible from on-chain data.

2. The deployed programs

RoundFi is four programs. They are deployed to Solana devnet at the addresses below and are **shared across clusters** (the same program IDs are the mainnet target), so a verifier can inspect them directly on a block explorer today.

Program	Devnet ID
roundfi-core	8LVrgxKwKwqjcdq7rUUwWY2zPNk8anpo2JsaR9jTQQjw
roundfi-reputation	Hpo174C6JTCfiZ6r8VYVQdKxo3LBHaJmMbkgREkxe9R2
roundfi-yield-mock	GPTMPgxexhwhkXNovnfrcSsmoWPUhedvKAQfTV2Ef5AQ
roundfi-yield-kamino	74izMa4WzLuHvtzDLdNzcyygKe5fYwtD95EiWMuzhFdb

The split is deliberate. **roundfi-core** holds all pool and protocol logic — lifecycle, the four vaults, the contribution split, the Triple Shield, the Escape Valve, the pause circuit-breaker. **roundfi-reputation** holds the score ladder and attestation surface, kept as a separate program precisely so that reputation is portable and not entangled with any one pool. The two yield adapters are **pluggable**: `roundfi-yield-mock` is the devnet adapter exercised below; `roundfi-yield-kamino` is the mainnet target (its on-chain CPI is shipped, with the operational reserve pin still pending).

3. The validated capabilities

Nine capability rows, mapped to the three pools they were exercised on. Pools are identified by their on-chain address prefix and a short ordinal (43 / 44 / 45) for legibility.

#	Capability	Pool	Status
1	Full lifecycle + Pass-3 reputation scoring	Ga2RwgSk... (43)	
2	<code>close_pool</code> (terminal Closed)	Ga2RwgSk... (43)	
3	Yield Cascade (init → deposit → harvest)	4SZCKeQL... (44)	
4	Escape Valve — direct (list → buy)	4SZCKeQL... (44)	
5	Pause circuit-breaker (pause → gate → unpause)	—	
6	Escrow vesting (<code>release_escrow</code> , on-time)	4SZCKeQL... (44)	
7	Escape Valve — commit-reveal (anti-MEV)	4SZCKeQL... (44)	
8	Rent-reclaim ceremony (<code>close_member</code> → <code>close_pool_vaults</code>)	Ga2RwgSk... (43)	
9	Default settlement (<code>settle_default</code>)	Hg9AkTCg... (45)	armed; grace-gated

What each row exercised, and why it matters:

3.1 Full lifecycle + Pass-3 scoring — pool 43 (Ga2RwgSk...)

A pool driven through its entire state machine: `Forming` → `Active` → `Completed`, with members joining, contributing, and claiming their carta each cycle. The load-bearing part is **Pass-3 reputation scoring**: this run is where the "received vs kept" separation was verified on-chain. Schemas `PAYMENT` (1), `PAYOUT_CLAIMED` (6), and `POOL_COMPLETE` (4) emitted **distinctly**, with `pay_out_claimed` landing in the **neutral** bucket — proving that *receiving* your carta is score-neutral, while the `+50` and the completed-pool bump fire only on the member's final contribution. That is RoundFi's entire thesis, demonstrated live.

3.2 `close_pool` — pool 43

The `Completed` → `Closed` transition. A pure terminal state flip that decrements committed TVL and **moves no funds** — distinct from the rent reclaim in §3.8, which is what actually empties the pool.

3.3 Yield Cascade — pool 44 (4SZCKeQL...)

The full idle-float yield path: `init` → `deposit` → `harvest`. Idle USDC was parked into the yield adapter (`deposit_idle_to_yield`) and the surplus harvested (`harvest_yield`), exercising the waterfall that splits realized yield across the protocol fee, the guarantee fund, the LP slice, and the participants' residual.

3.4 Escape Valve, direct — pool 44

The secondary-market exit in its simplest form: `escape_valve_list(price)` → `escape_valve_buy`. A member sold their **position** (slot + NFT + pending obligations); the buyer atomically paid the seller, inherited the slot's operational state, and received the re-frozen position NFT. The seller keeps their own wallet-bound reputation — only the slot's obligations transfer.

3.5 Pause circuit-breaker — protocol-wide (no single pool)

`pause` → `gate` → `unpause`. With the protocol paused, gated instructions revert with `Protocol - Paused`; unpausing restores them. This is protocol-level state, not tied to one pool, which is why its pool column is `-`. `settle_default` is a deliberate carve-out — a default in flight must stay settleable even while paused.

3.6 Escrow vesting — pool 44

`release_escrow` on an **on-time** member. This is the mirror of default seizure: a member who pays on time progressively releases their staked USDC from escrow in tranches. Pay on time, get your stake back; pay late, it stays locked.

3.7 Escape Valve, commit-reveal (anti-MEV) — pool 44

The MEV-resistant listing path: a commit hides the price behind a hash, a reveal publishes it and arms a short cooldown, and the buyer — who already knows the price off-chain — lands their

purchase at the cooldown boundary, ahead of any searcher reacting to the now-public price. The anti-front-running design, exercised live.

3.8 Rent-reclaim ceremony — pool 43 (the true end of the lifecycle)

This is the row a skeptic should look at hardest. `close_pool` (§3.2) only flips the status to `Closed`; it does **not** empty the pool. The **rent-reclaim ceremony** is what actually winds a pool down: `close_member` is called once per member (returning each member's rent to them), then `close_pool_vaults` sweeps residual USDC to the treasury, closes the four vault ATAs and the Pool PDA, and returns that rent to the authority.

The rent-reclaim ceremony is the TRUE end of a pool's lifecycle. The proof it worked is economic: after the ceremony on pool 43, the **authority's SOL balance went up — +0.0108 net of fees**. A pool that can be fully unwound, returning more rent than it costs to close, is a pool that genuinely existed and was genuinely cleaned up. Nothing was left stranded on-chain.

3.9 Default settlement — pool 45 (Hg9AkTCg...), armed and grace-gated

The one capability **not yet run to completion**, and stated as such. Pool 45 exists with a member set up to default, but `settle_default` is **time-gated**: it cannot fire until the grace window after the missed cycle has elapsed. The status is `armed; grace-gated` — the scenario is staged and the instruction is ready; only wall-clock time stands between here and the replay. The grace window is generous by design: default is a last resort, not a tripwire.

4. Why "seven of eight" is the honest count

The summary line is: **seven of the eight protocol capability areas were exercised end-to-end on devnet on 2026-06-12 with real USDC; the eighth (default settlement) is armed and time-gated.** We state it that way on purpose. The default-settlement scenario is real and staged on pool 45 — what is missing is only the passage of the grace period, not any code or setup. Reporting it as "armed; grace-gated" rather than padding the count is the same discipline applied throughout RoundFi: we do not claim what we cannot show.

Everything else — the full pool lifecycle, Pass-3 scoring, the Yield Cascade, both Escape Valve paths, escrow vesting, the pause breaker, and the complete rent-reclaim wind-down — ran to completion against live programs with real token transfers.

4.1 What the count does *not* include yet

The **contemplation subsystem** — sorteio ordering plus all three lance phases (prepayment, embedded bid, sealed free bid) — shipped to devnet in **July 2026**, after the 2026-06-12 exercise this document reports. It is **deployed and covered by proofs**, but those proofs are **in-process lifecycle specs** (`litesvm` driving whole pools: `litesvm_sorteio_draw`, `litesvm_ordering_policy`, `litesvm_prepay_ahead`, `litesvm_lance_embutido`, `litesvm_lance_livre`, plus a byte-for-byte TS ↔ Rust commit-hash parity suite) rather than a devnet run with real USDC under the v5.2 runbook shape.

By the standard this document holds itself to, that is **not** the same evidence class as the eight rows above, so it does not get a row. It earns one when the pools driving it are replayed on devnet and the signatures land in the runbook.

Separately: external review of the lance security design remains an **open mainnet gate** — see [MAINNET_READINESS.md](#) item 1.12.

5. How to verify this yourself

This document deliberately contains **no transaction signatures**. The signatures, together with the reproducible runbook, live in [docs/operations/v52-devnet-runbook.md](#). The intended verification path:

1. Inspect the four program IDs from §2 directly on a devnet block explorer.
2. Pick any capability row from §3 and note its pool (Ga2RwgSk... / 4SZCKeQL... / Hg9AkTCg...).
3. Pull the matching signature from the runbook and confirm the on-chain transaction — the USDC transfers, the emitted attestations, the state transitions — against the claim.

The point of keeping signatures in the runbook and capabilities here is separation of concerns: this page tells you **what is true**; the runbook lets you **reproduce it**.

Cross-references: protocol mechanics exercised by these capabilities → [02-technical-whitepaper](#); program topology → [03-architecture-spec](#); the Pass-3 scoring proven in row 1 → [04-behavioral-reputation-score](#). Full deployed state and the capability matrix live in MASTER-SPEC §11; the contribution split, Triple Shield, and Escape Valve mechanics in §4.